



GK Master

Train your knowledge. Every single day.

PROJECT REPORT

GK Master is an offline-first Progressive Web App for daily general-knowledge practice. It runs entirely in the browser with no backend, no account and no running cost, installs to a phone Home Screen, and continues to work with no network connection. This report documents the requirements it was built against, the architecture and engineering decisions behind it, how it was built and verified, and what it does not yet do.

VERSION	v1.1.0
AUTHOR	Gopi Patel
REPORT GENERATED	Wed, 23 Sep 2026 17:20:14 GMT
APPLICATION TYPE	Installable Progressive Web App (static, client-side only)
TECHNOLOGY	TypeScript, React 18, Vite 5 - no backend, no UI framework
QUESTION BANK	602 built in, +10 in 1 downloadable pack
CODEBASE	47 source files, 7,559 lines (excluding question data)
AUTOMATED TESTS	166 tests across 8 suites, all passing
RUNTIME DEPENDENCIES	4
BUILD OUTPUT	19 static files

How claims in this report are qualified

Statements about behaviour are separated from statements about verification. Where something was tested in a real browser, the section says so and gives the observed result. Section 19 states the limitations, including what is not implemented and what was not tested.

Contents

#	Section	Contents
02	Project Overview	What the product is, who it serves, and its operating context
03	Project Objectives	The measurable goals the build was held to
04	Problem & Requirements	The originating problem and the requirements derived from it
05	Solution Overview	The shape of the solution and why this approach was chosen
06	Key Features	Delivered capabilities, shown in the running application
07	System Architecture	Layering, module responsibilities and data flow
08	Technology Stack	Every dependency, its version, and why it is present
09	Core Workflows	Daily challenge, revision, pack updates and PDF import
10	Engineering Decisions	Problem, decision, implementation and result for each
11	AI-Assisted Development	How the work was produced, directed and validated
12	Implementation Details	Question bank, learning engine, PWA layer, storage
13	Challenges & Solutions	Defects found during development and how they were resolved
14	Testing & Verification	Suite coverage and browser verification results
15	Reliability & Data Integrity	Failure handling, validation boundaries, privacy
16	Deployment & Usage	Build output, hosting configuration, installation
17	Role & Contribution	Ownership, direction and validation
18	Results & Verified Metrics	Measured outcomes against the objectives
19	Limitations	Not implemented, and not tested
20	Future Improvements	Practical next steps from the current state
21	Conclusion	Summary of outcome
22	Master Prompt	A single prompt to rebuild the project, in six prompt structures

Project Overview

2.1 What the product is

GK Master is a personal general-knowledge trainer. The user opens a URL, adds it to their phone Home Screen, and practises a short set of questions each day. The application selects what to ask based on which topics the user is weakest at, returns questions they answered incorrectly on a widening schedule, and explains every answer so a mistake teaches something rather than only costing a point.

2.2 Intended user and context of use

Attribute	Detail
Primary user	An individual studying general knowledge, typically for competitive examinations
Primary device	iPhone, used in short sessions
Connectivity	Often intermittent or absent; the application must not depend on a network
Session length	A few minutes daily, sustained over weeks rather than one long sitting
Cost tolerance	Zero. No subscription, no App Store fee, no hosting bill

2.3 Operating context

The application is a static site. There is no server process, no database and no authentication. All user data lives in the browser on the device that produced it. This shapes every subsequent decision in this report: features that would normally be delegated to a backend, such as content updates and progress storage, had to be solved within the browser.

Project Objectives

The build was held to the following objectives. Section 18 reports the measured outcome against each.

#	Objective	How it is measured
O1	Usable as an installed app on iPhone without native development	Installs via Safari Add to Home Screen; launches standalone
O2	Fully functional with no network connection	App loads and quizzes run with the server stopped
O3	A question bank large enough for sustained daily use	At least 500 questions, each with a verified answer and explanation
O4	Practice that adapts to the user rather than being random	Weak categories receive a measurably larger share of questions
O5	A daily challenge that is stable within a day	Same date produces an identical set; a new date produces a different one
O6	Statistics that are never misleading	No percentage shown below a minimum sample size
O7	The bank can grow after release without rebuilding the app	New questions installable from a pack or a PDF at runtime
O8	Zero running cost and no user account	No backend, no paid service, no credentials in the repository
O9	Correctness demonstrated, not asserted	Automated test suite plus browser verification of each claim

Problem & Requirements

4.1 The problem

Daily general-knowledge practice tools generally fall into two groups. Web quizzes require a connection and lose progress between visits. Native applications require an App Store account and ongoing cost, and many fund themselves through advertising or subscription. Neither suits a single user who wants to practise on a phone, frequently offline, without paying or creating an account.

A further problem is pedagogical rather than technical. A quiz that asks random questions and reports a score does not help someone improve: it repeats what the user already knows and does not bring back what they got wrong. The requirement was therefore not only "a quiz app" but a practice tool that targets weakness and schedules revision.

4.2 Functional requirements

ID	Requirement	Status
F1	Daily challenge of configurable length (5, 10, 20 or 30 questions)	Delivered
F2	The daily set is identical on re-opening the same day, and resumable mid-quiz	Delivered
F3	Immediate answer feedback with an explanation for every question	Delivered
F4	Category practice across 14 defined subject areas	Delivered
F5	Additional modes: random, weak areas, revision, quick (5), deep (50)	Delivered
F6	Spaced revision of incorrectly answered questions	Delivered
F7	Progress tracking: accuracy, per-category mastery, difficulty split, streaks	Delivered
F8	Settings: question count, theme, feedback options	Delivered
F9	Export, import and reset of all user data	Delivered
F10	Add new questions after release without rebuilding the application	Delivered
F11	Import multiple-choice questions from a PDF question paper	Delivered

4.3 Non-functional requirements and constraints

ID	Constraint	Consequence for the design
N1	No native application or App Store distribution	Progressive Web App, installed from the browser
N2	No backend, database or serverless function	All logic and storage run client-side
N3	No paid API, key or subscription	Question content is authored, not fetched from a service
N4	No account or login	No user identity exists anywhere in the system
N5	No analytics, advertising or third-party requests	System fonts, inline SVG icons, no CDN
N6	Must work offline after first load	Service worker precaches the shell and question bank
N7	iPhone is the primary target	Safe-area insets, standalone display, portrait, large touch targets

N8

Deployable to any static host, including a sub-path

Relative asset base and hash-based routing

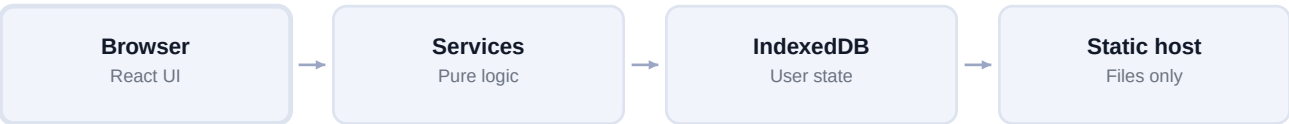
Solution Overview

5.1 Approach

The solution is a single-page application compiled to static files, with three layers of behaviour that would ordinarily require a server, implemented instead in the browser:

- Content delivery - the question bank ships inside the bundle, and can be extended at runtime through downloadable packs or PDF import, both validated client-side.
- Persistence - IndexedDB stores settings, per-question learning state and session history, with an in-memory fallback when it is unavailable.
- Offline capability - a hand-written service worker precaches the application shell and question data, with a content-derived cache version so updates retire the previous cache cleanly.

5.2 High-level structure



No server-side component exists at any point in the flow.

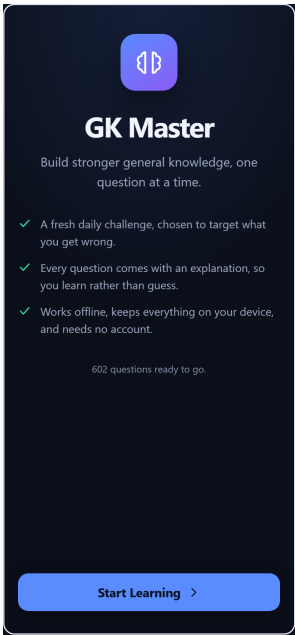
5.3 Why this approach

Alternative considered	Why it was not used
Native iOS application	Requires a paid developer account and App Store review; the user explicitly excluded both. A PWA installs to the Home Screen and meets the same day-to-day need.
Backend with a question API	Introduces hosting cost and a network dependency, directly conflicting with the offline and zero-cost requirements.
Third-party quiz or trivia API	Paid, network-dependent, and gives no control over question quality or explanations.
Hosted database for progress	Would require accounts and introduce privacy obligations for data that only matters on one device.

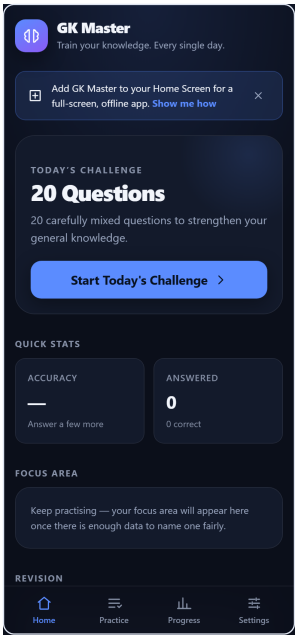
Key Features & Capabilities

Every screenshot in this section was captured from the production build running in a real Chromium browser at an iPhone viewport (390 x 844 at 2x device scale), by driving the application as a user would. None are mockups. They are regenerated by scripts/captureScreens.mjs.

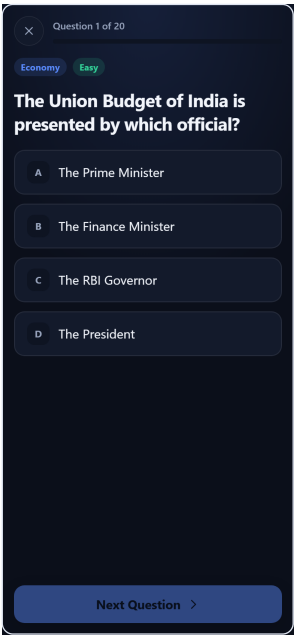
6.1 Onboarding and the daily challenge



First launch. No account, no sign-up.



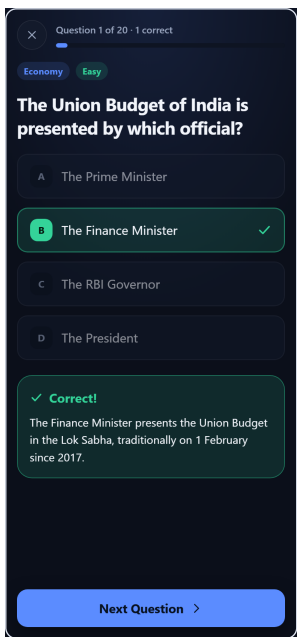
Home on day one. Statistics are withheld rather than shown as zero.



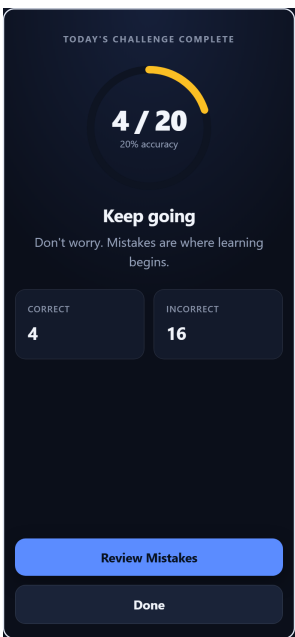
A question. Category and difficulty are shown; options are large touch targets.

First launch requires no registration. The Home screen leads with the daily challenge and shows statistics only where the data supports them - on day one, accuracy is shown as a dash rather than zero.

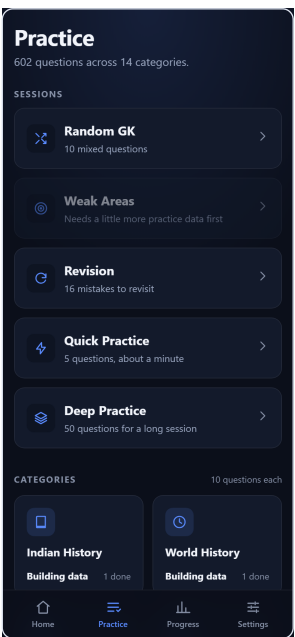
6.2 Answering and completion



After answering. Correctness is shown by colour, icon and text together.



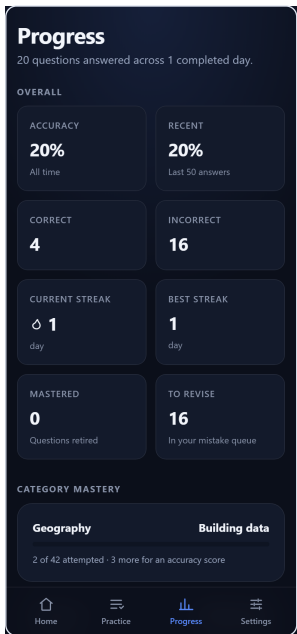
Session results, with an encouraging message banded by score.



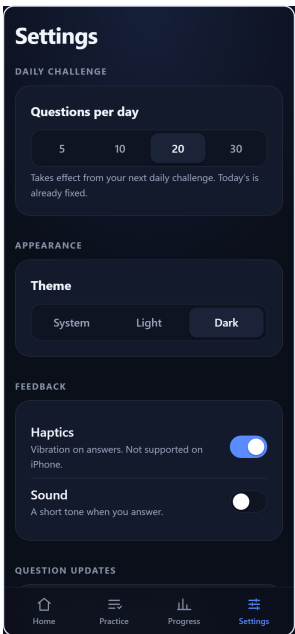
Practice modes and the category grid.

Selecting an answer locks all options, reveals the correct one and shows the explanation immediately. Correctness is signalled by colour, an icon and text together, so it never depends on colour alone. The results screen bands its closing message by score and offers a review of the mistakes.

6.3 Progress, settings and theming



Progress. Categories without enough data are labelled, not scored.



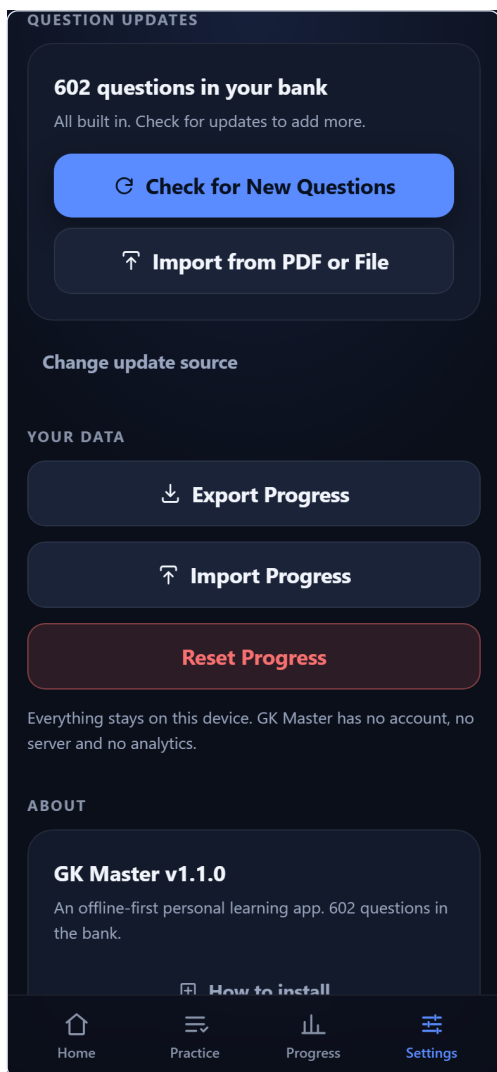
Settings: daily length, theme, feedback and data controls.



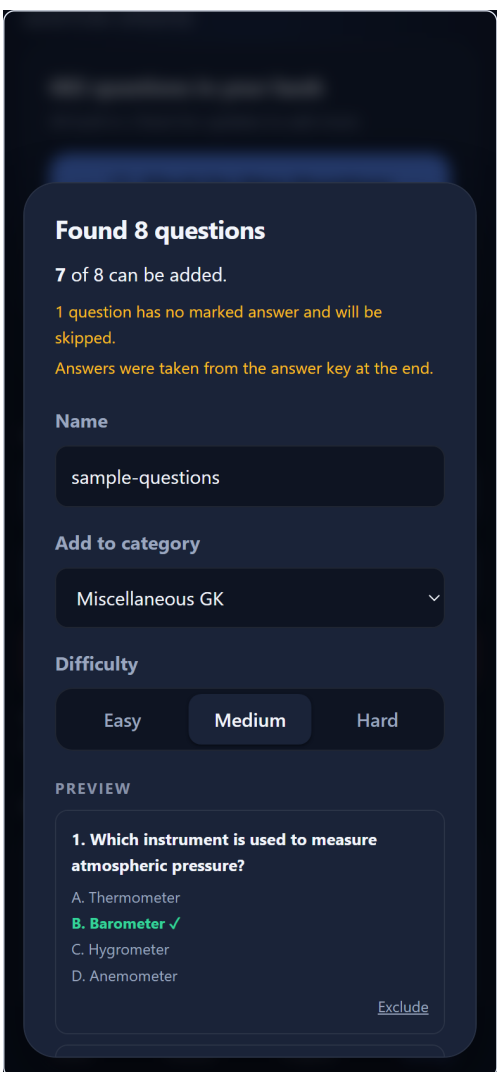
The same Home screen in the light theme.

Progress reports overall and recent accuracy, per-category mastery, difficulty performance and streaks. Categories without enough data are labelled rather than scored. Both themes are complete palettes, not a filter.

6.4 Extending the question bank



Question updates: check a source, or import from a PDF.



PDF import review. Nothing is added until the parse is confirmed.

Questions can be added after release in two ways: by installing a published pack from a configurable source, or by importing a PDF question paper. The PDF path shows a review screen listing exactly what was parsed before anything is added to the bank.

System Architecture

7.1 Layering

The central architectural decision is that every algorithm lives in a React-free service module. Selection, scoring, spaced repetition, statistics, streaks, storage and MCQ parsing contain no React imports and no DOM access. This is why 154 of the 166 tests run without rendering anything, and why each algorithm can be reasoned about and tested in isolation from the interface.

Layer	Responsibility	React-free	Tested directly
data	Question bank loading, validation, indexing	Yes	Yes
services	All algorithms and persistence	Yes	Yes
hooks	Binding services to React state	No	Via flow tests
pages / features	Screens and interaction	No	Via flow tests
components	Reusable presentational primitives	No	Via flow tests

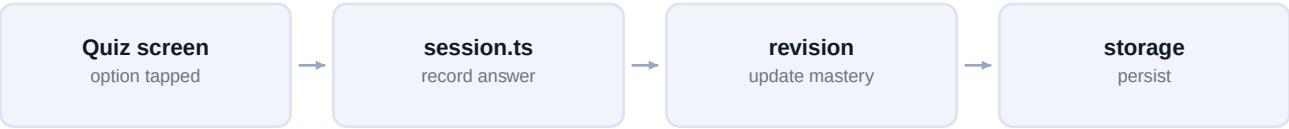
7.2 Directory layout

```

public/
  icons/          5 generated PNG icons + favicon.ico
  packs/          published question packs and their index
  sample-questions.pdf  example MCQ paper used to exercise import
  manifest.webmanifest
src/
  components/     icon set, UI primitives, bottom navigation
  features/quiz/  quiz screen, answer options, results
  features/packs/ question updates and PDF import UI
  pages/          Home, Practice, Progress, Settings, Install, Welcome, Quiz
  data/
    questions/*.json  bundled question bank, one file per category
    questionBank.ts   loads, validates and indexes bundled + pack questions
  services/
    quiz/selection.ts adaptive and deterministic question selection
    quiz/session.ts   session lifecycle and scoring
    revision/         spaced repetition
    statistics/        accuracy, category and difficulty statistics
    streak/           daily streak rules
    storage/          IndexedDB wrapper, backup and restore
    packs/            pack validation, install, update checks
    import/           PDF text extraction and MCQ parsing
  hooks/          application store, theme, install prompt
  sw/service-worker.js  service worker source, templated at build time
  styles/         design tokens, global and component CSS
  scripts/        validation, icons, packs, preview, capture, report
  tests/          8 Vitest suites

```

7.3 Data flow through a single answer



Each answer is persisted as it happens, so closing the app mid-quiz loses nothing.

7.4 Code size

Area	Files	Lines
Application source (excluding question data)	47	7,559
Question data (JSON)	14	12,692
Tests	8	2,028
Build and tooling scripts	12	3,605

Technology Stack

8.1 Runtime dependencies

The shipped application depends on 4 runtime packages. State management, styling, icons, storage, routing behaviour and the service worker are implemented directly in the project rather than taken from libraries.

Package	Version	Purpose and rationale
pdfjs-dist	6.3.289	PDF text extraction for the question importer. Loaded lazily and excluded from the offline precache, so it costs nothing unless used.
react	18.3.1	UI rendering. Chosen for its component model and mature testing tooling.
react-dom	18.3.1	DOM renderer for React.
react-router-dom	6.30.6	Routing. Hash-based, so the build runs from any host and any sub-path without server rewrite rules.

8.2 Development dependencies

Package	Version	Purpose
@testing-library/jest-dom	6.9.1	Additional DOM assertions.
@testing-library/react	16.3.2	Renders and queries components the way a user would.
@testing-library/user-event	14.6.5	Realistic interaction simulation.
@types/node	26.2.0	Node type definitions for build scripts.
@types/react	18.3.31	React type definitions.
@types/react-dom	18.3.7	React DOM type definitions.
@vitejs/plugin-react	4.7.0	React fast refresh and JSX transform.
fake-indexeddb	6.2.5	Real IndexedDB semantics in tests, so storage is exercised rather than mocked.
jsdom	25.0.1	DOM implementation for component tests.
pdfkit	0.20.2	Generates this report. Tooling only, never shipped to the browser.
puppeteer-core	25.10.0	Drives an installed Chrome to capture screenshots. Uses the existing browser, so no Chromium download.
tsx	4.23.12	Runs the TypeScript validation script without a separate compile step.
typescript	5.9.3	Static typing across application, tests and build scripts.
vite	5.4.21	Build tool and development server.
vitest	2.1.9	Test runner sharing the Vite pipeline, so tests resolve modules identically to the build.

8.3 Deliberately excluded

Not used	Reason
CSS framework	A design-token system gives full control of the intended look with no build complexity and no unused CSS.
State management library	Application state is small and mostly persisted; one React context is sufficient.
Service worker toolkit	The caching rules are simple. A hand-written worker plus a small Vite plugin gives exact control of precache contents and cache versioning.
Icon library	Icons are inline SVG paths defined in the project: they inherit colour, add no request and work offline.
Backend or database	Not required. All state is per-device and fits in IndexedDB.
LLM or paid API for content	Would require a key and a server. Questions are authored and fact-checked instead.

Core Workflows

9.1 Daily challenge



On opening, the application looks for an existing session for the current local date. If one exists it is resumed at the question the user reached; otherwise a new set is generated from a seed derived from the date. Completing the challenge records the day against the streak.

9.2 Revision



An incorrect answer places a question in the mistake queue and schedules it for the following day. Each subsequent correct answer advances its mastery level and lengthens the interval. At the top level the question retires from the queue permanently, so the queue cannot grow without bound.

9.3 Question pack update



9.4 PDF import



Parsing a question paper is best-effort by nature, so the review step is mandatory rather than optional: nothing reaches the bank until the user has seen what was found, including which questions were skipped and why.

Important Engineering Decisions

10.1 Deterministic daily challenge

Problem	A daily challenge must be the same set of questions every time it is opened on a given day, and a different set the next day. Ordinary randomness gives a new set on every render, which would make progress meaningless.
Decision	Derive selection from a seeded pseudo-random generator keyed on the local calendar date, rather than storing a pre-generated list or using Math.random.
Implementation	The date string is hashed with FNV-1a and used to seed a Mulberry32 generator. Category quotas are allocated with the largest-remainder method, which is deterministic for a given set of weights. Local dates are used throughout rather than UTC, so a session started late in the evening belongs to that day.
Result	Verified by test: the same date produces an identical set, a different date produces a different one. The session is also persisted, so a part-finished challenge resumes rather than regenerating.

10.2 Adaptive selection that never abandons a category

Problem	Practice should concentrate on weak topics, but a naive "drill the weakest" rule collapses the question mix onto one or two categories and the user stops seeing broad general knowledge.
Decision	Weight categories continuously by accuracy with a floor, rather than filtering to the weakest.
Implementation	Weight = $1 + \max(0, 70 - \text{accuracy}) / 70 \times 2$. A category at 0% accuracy receives three times the weight of one at 70% or above, and no category ever reaches zero. Untouched categories receive a small boost so the bank is explored. Difficulty mix shifts from 50/35/15 to 20/45/35 as recent accuracy improves.
Result	Tested: with one category at 20% accuracy and the rest at 97%, the weak category takes a measurably larger share of a 40-question selection while every other category still appears.

10.3 Withholding statistics rather than showing zero

Problem	Accuracy computed from one or two answers is noise. Showing "0%" for a category after a single wrong answer is both discouraging and factually misleading.
Decision	Make accuracy a nullable value throughout the domain model and render an explicit state instead of a number when the sample is too small.
Implementation	Below 5 attempts a category reports "Building data" with how many more are needed; at zero attempts it reports "Not started". A category needs at least 10 attempts before it can be named the weak area. The type system carries "number null" so a missing value cannot be silently formatted as zero.
Result	Guarded by tests. A new user sees dashes and labels rather than a wall of zeros, and the focus area appears only once it is statistically meaningful.

10.4 Hand-written service worker

Problem	Offline support requires precise control over what is cached and when caches are retired. A stale service worker or cache is one of the most common ways an installed PWA becomes permanently out of date.
Decision	Write the service worker directly and template it at build time, rather than adopting a caching toolkit.

Implementation	A 45-line Vite plugin walks the build output, injects the precache manifest, and derives the cache name from the SHA-256 content hashes of every precached file. Navigations are network-first with a cache fallback; content-hashed assets are cache-first. Precache entries are added individually so one failed asset cannot leave the app without a shell.
Result	Any change to any shipped file produces a new cache name and the previous cache is deleted on activation. Verified in the browser: the old cache was removed on rebuild and the app ran with the server stopped.

10.5 Lazy-loading the PDF reader

Problem	pdf.js is approximately 1.7 MB. Including it in the offline precache would roughly triple what every user downloads to install the application, for a feature most users will never open.
Decision	Load the PDF reader only when a PDF is actually selected, and exclude it from the precache manifest.
Implementation	A dynamic import pulls the library on demand. The build plugin filters PDF chunks out of the precache list. The service worker still caches it at runtime on first use, so a second import works offline.
Result	Initial download stays small: 717.3 KB total gzip across the build, of which the PDF reader is fetched only on demand.

10.6 Relative base and hash routing

Problem	The deployment target was undecided. Path-based routing breaks on static hosts without rewrite rules, and absolute asset paths break when a site is served from a sub-path.
Decision	Build with a relative asset base and route on the URL hash.
Implementation	Vite base is set to "/" and the router is a HashRouter. The service worker registration resolves its URL relative to the document rather than the origin root.
Result	Verified by serving the same build under a /gk-master/ sub-path: all assets resolved, the manifest loaded and the service worker scoped correctly, with no server configuration.

SECTION 11

AI-Assisted Development

This project was built using AI-assisted development. The implementation was produced by an AI coding agent (Claude, running in an agentic coding environment with direct file-system and shell access) working under my direction. This section describes that process accurately, because the distinction between what I directed and what the tool produced is material to reading the rest of this report.

11.1 Division of responsibility

Activity	Owned by me	Performed by the AI agent
Product definition and requirements	Defined the product, constraints and acceptance bar	Restated requirements and flagged conflicts
Scope decisions	Decided what was in and out at each stage	Proposed options with trade-offs
Technology selection	Set the hard constraints that ruled options in or out	Selected specific libraries within those constraints and justified each
Implementation	Reviewed and accepted or redirected	Wrote the source, tests, tooling and question content
Content accuracy	Required primary-source verification	Checked facts against sources and dropped unverifiable items
Verification	Required evidence rather than assertion	Ran tests and drove the app in a browser, reporting observed results
Direction changes	Redirected the work when the output did not fit the need	Re-planned and implemented the revised approach

11.2 How direction changed the outcome

Several parts of the delivered system exist because of decisions taken during the work rather than in the original brief. These are documented because they show where judgement was applied:

- Native packaging was raised and rejected. After reviewing what an iOS build would require - a paid developer account, a Mac, and re-signing every seven days for the free route - the decision was to stay with a Progressive Web App and invest in the install experience instead.
- The deployment target was changed to Vercel mid-project, with the requirement that the project remain a plain static site with no server-side mechanism. This produced the hosting configuration in Section 16.
- PDF import was added as a requirement after the pack system was working, on the basis that question banks in practice circulate as PDF question papers rather than as JSON.
- The requirement that current-affairs content be verified against primary sources, rather than recalled, came from the direction that the report and the app must not state anything unverified.

11.3 Controls applied to AI-produced work

AI-generated code and content were not accepted on trust. The following controls were applied, and each of them caught at least one real defect, documented in Section 13:

Control	What it caught
Automated test suite	Regressions in selection, parsing and storage as features were added
Question bank validator	Duplicate text, malformed records, and a severe answer-position bias

Browser verification of every claim	A service-worker MIME failure that unit tests could not surface
Measured rather than assumed metrics	Contrast ratios below the accessibility threshold in both themes
Generated statistics for this report	Prevents documented figures from drifting from the codebase

Accuracy of this section

The code in this repository was written by an AI agent under direction, not typed by hand. It is recorded that way here deliberately. What is claimed as my contribution is the requirements, the decisions, the redirection, the acceptance criteria and the validation - which Section 17 sets out in full.

Implementation Details

12.1 Question bank

602 questions ship inside the application across 14 categories, with 10 more available as a downloadable pack. Every question carries exactly four options, one verified answer and a written explanation.

Category	Count	Category	Count
Current Affairs	52	India GK	42
Miscellaneous GK	44	Indian History	42
Science	44	Indian Polity	42
Art & Culture	42	Sports	42
Books & Authors	42	Technology	42
Economy	42	World GK	42
Geography	42	World History	42

Difficulty and answer-position distribution

Difficulty	Count	Share	Answer position	Count	Share
Easy	235	39.0%	Option A	151	25.1%
Medium	282	46.8%	Option B	151	25.1%
Hard	85	14.1%	Option C	150	24.9%
			Option D	150	24.9%

Schema and validation gates

```
{
  "id": 6001,                // unique integer, range reserved per category
  "question": "Which planet is known as the Red Planet?",
  "options": ["Venus", "Mars", "Jupiter", "Mercury"],
  "correctAnswer": 1,        // zero-based index into options
  "explanation": "Mars appears reddish because iron minerals have oxidised.",
  "category": "Science",     // one of 14 known categories
  "difficulty": "Easy",      // Easy | Medium | Hard
  "tags": ["astronomy"],
  "source": "GK Master editorial",
  "sourceUrl": null,
  "publishedDate": null      // required for Current Affairs
}
```

The validator fails the build on any of the following:

- Duplicate ids, or duplicate question text after whitespace and case normalisation.
- Any question without exactly four non-empty, non-duplicate options.
- A correctAnswer index outside 0-3, or an explanation shorter than 20 characters.
- An unknown category or difficulty value.
- A Current Affairs question without a valid publishedDate.

- Answer-position skew above 35% for any single option.
- A total below the 500-question minimum.

Current affairs handling

Static content is never presented as live news. Every Current Affairs question carries a `publishedDate` and the quiz screen displays the month and year alongside it. Coverage runs to August 2026. Facts were checked against primary sources - pmindia.gov.in and pib.gov.in for Indian government schemes, and dedicated encyclopaedia articles for international events. Two candidate facts that could not be confirmed against a second source were dropped rather than included.

12.2 Learning engine

Mastery level	0	1	2	3	4	5
Days until review	0	1	3	7	16	35

- A correct answer advances one mastery level; an incorrect answer drops back two, so a lapse is re-drilled soon without discarding accumulated history.
- A missed question always returns the next day, never the same day, so a single session cannot drill it repeatedly.
- Reaching the top level retires a question from the mistake queue permanently.
- The revision queue is ordered by urgency: overdue first, then repeat offenders, then most recently missed.
- Up to 25% of a daily challenge is reserved for revision questions that are genuinely due.

12.3 Progressive Web App layer

Concern	Implementation
Manifest	standalone display, portrait orientation, relative scope and <code>start_url</code> , 192 and 512 px icons in both "any" and "maskable" purposes
Navigations	Network-first with cache fallback, so an updated shell is picked up quickly but the app still opens offline
Static assets	Cache-first; filenames are content-hashed so a cached copy is always correct
Uncached and offline	Returns HTTP 504 rather than substituting the HTML shell, which would produce a misleading MIME error
Cache versioning	Derived from content hashes of all precached files; old caches deleted on activation
iPhone specifics	<code>viewport-fit=cover</code> with safe-area insets, <code>apple-touch-icon</code> , black-translucent status bar, 48px minimum touch targets

12.4 Storage

Object store	Key	Contents
kv	name	Settings and streak state
progress	questionId	Attempts, correct, incorrect, streak, mastery level, next review date
sessions	id	Quiz history, answers and timings, indexed by date
packs	id	Installed question packs and their questions

12.5 Design system and accessibility

All colour, spacing, typography, radius and motion values are CSS custom properties defined once; components reference tokens only, so light and dark are the same components with a redefined palette. Typography uses the system font stack, so no web font is downloaded.

Contrast was measured in the running application rather than assumed. The first measurement found several pairs below the WCAG AA threshold of 4.5:1; the palette was corrected and re-measured.

Colour pair	Dark theme	Light theme
Primary text on background	17.52:1	16.65:1
Secondary text on surface	7.09:1	6.22:1
Tertiary text on surface	5.56:1	5.23:1
Accent on background	6.05:1	5.65:1
Success on surface	9.03:1	5.16:1
Error on surface	6.27:1	4.55:1
Warning on surface	10.39:1	5.38:1
Button label on primary	5.98:1	6.06:1

- Correctness is conveyed by colour, icon and text simultaneously, so it never depends on colour alone.
- Focus moves to the question heading on each new question, so screen-reader users hear the new question.
- Explanations are announced through an aria-live region when an answer is locked in.
- Semantic HTML with labelled controls, radiogroups for segmented choices and role="switch" for toggles.
- prefers-reduced-motion collapses all animation to near-zero duration.

Challenges & Solutions

Each of the following was found during development and fixed. They are recorded because they show where the difficulty in the project actually lay, and because several were caught only by deliberately checking rather than assuming.

13.1 Answer-position bias made the app beatable

Problem	The first draft of the question bank had 92.7% of correct answers at option B. A user could score 93% by always selecting B without reading the question, which defeats the purpose of the application entirely.
Decision	Treat answer-position distribution as a correctness property of the data, enforced automatically rather than left to authoring discipline.
Implementation	A rebalancing tool swaps each correct option into a target position using a single swap per question, so the remaining options keep their authored order. The validator fails any bank where one position exceeds 35%, and a unit test enforces the same bound.
Result	Distribution is now 25.1% / 25.1% / 24.9% / 24.9%. Verified in the running app: always choosing option B scored 22%, which is chance.

13.2 PDF questions silently lost after the fourth

Problem	Importing a question paper returned only 4 of 7 questions, with no error. After four options labelled A to D, a question numbered "5." has the same sequence index a fifth option would, so it was absorbed into the previous question.
Decision	Distinguish option runs by their numbering style, not only by sequence position.
Implementation	Each option run records whether it uses letters or digits. A numeric marker cannot continue a run that began with letters, so "5." after options A-D is recognised as a new question. A regression test covers the exact layout that failed.
Result	All 7 questions are now found. The defect was invisible to the user, which is why the review screen showing the parse result before import matters.

13.3 A page header became a question explanation

Problem	On a two-page PDF, the running header was captured as the explanation text of the question that preceded the page break. Header detection required a line to repeat three or more times, which a two-page document cannot satisfy.
Decision	Detect page furniture by position within a page rather than by repetition count.
Implementation	The extractor separates pages with a form feed. Only lines appearing at the top or bottom edge of two or more pages are treated as furniture, so a legitimate repeated option such as "None of these" in the body survives.
Result	Verified by re-importing the sample paper: the affected question now carries a correct explanation, and a test covers both the header case and the false-positive case.

13.4 Service worker rejected, with a misleading error

Problem	PDF import failed with "That file could not be read as a PDF". The real cause was that the preview server served .mjs as application/octet-stream, so the browser refused the worker module. The error message pointed at the wrong thing.
Decision	Remove the dependency on host MIME behaviour, and make the error state what actually failed.

Implementation	The worker is now bundled through Vite's worker import rather than referenced by URL, so it no longer depends on the host serving .mjs correctly. The preview server declares the correct types, and worker failures are reported distinctly from malformed files.
Result	Import works, and a future failure of this kind will name its own cause rather than blaming the input.

13.5 Inconsistent revision counts between screens

Problem	Immediately after answering 19 questions incorrectly, the Home screen reported nothing due for review while the Practice screen listed all 19. Both were reading the same data with different rules.
Decision	Separate what the spaced schedule governs from what the user is permitted to do.
Implementation	The spaced schedule now determines only what is folded into the daily challenge. Revision on demand offers the full outstanding queue, and both screens read that same figure.
Result	The two screens agree, and the user can revise when they choose rather than when the schedule allows.

13.6 Report layout defects

Problem	The first generated report ran to 65 pages, most of them blank, and 31 text blocks overflowed the right margin.
Decision	Check the generated document programmatically rather than trusting that it rendered correctly.
Implementation	The blank pages came from footer text triggering page breaks at the bottom margin; the overflow came from pdfkit retaining the horizontal cursor from the previous positioned element. Both were fixed at a single point in the layout engine, and scripts/checkReport.mjs now inspects every text item for margin overflow, unrenderable glyphs and empty pages.
Result	The report is checked on every regeneration and currently reports zero layout issues.

Testing & Verification

14.1 Automated test suites

166 tests across 8 suites, all passing. Storage is exercised against a real IndexedDB implementation rather than mocks.

Suite	Tests	Coverage
engine	38	Spaced-repetition intervals and mastery transitions, streak rules including staleness, statistics minimum-data guards, session scoring, seeded randomness, date arithmetic
mcqParser	30	Every supported PDF layout plus awkward cases: wrapped options, single-line options, numeric markers, running headers, answer keys, unanswered questions
selection	25	Daily-challenge determinism, category weighting, difficulty mix, quota allocation, recency avoidance, revision injection, short-pool behaviour
packs	22	Pack validation, install, duplicate rejection against the bundled bank, removal, re-registration at startup, remote update checks with the network stubbed
storage	15	IndexedDB round-trips, corrupted-record handling, settings repair, backup export and import including rejection of invalid files
importToPack	13	Converting parsed questions into an installable pack, and everything it refuses to import
flow	12	End to end: onboarding, a full daily challenge, answer locking, explanations, results, persistence across a remount, navigation, settings, reset confirmation
questionBank	11	Bank integrity: duplicate ids and text, option counts, answer indices, explanations, category coverage, answer-position balance

14.2 Browser verification

The following were confirmed by driving the running application, not by assertion:

Check	Observed result
Service worker registration	Activated and controlling the page
Offline operation	Server stopped entirely; app still loaded, ran quizzes and saved progress
Precache contents	All shell files cached, including index.html
Cache versioning	Previous cache deleted and replaced on rebuild
Manifest and icons	Manifest valid; all six icon files returned HTTP 200
Sub-path hosting	Served from /gk-master/; assets resolved and the worker scoped correctly
Persistence	Progress, streak and installed packs survived a full reload
Theme switching	Both themes applied, with the browser theme-colour meta updating
Answer-position bias	Always choosing option B scored 22%, i.e. chance
Backup export	Produced a valid JSON file with the expected structure and filename
Pack lifecycle	Discovered, installed, re-checked, reloaded, removed and reinstalled

14.3 Verification gate

A single command chains content validation, the full test suite and a production build. It is the pre-deployment gate.

```
npm run verify

PASSED - 602 questions validated
Test Files  8 passed (8)
   Tests   166 passed (166)
built in 16.11s
```

Reliability & Data Integrity

15.1 Trust boundaries

The application has no server, so its security surface is small, but it does accept data from outside itself: downloaded packs, imported PDFs and restored backups. Each is treated as untrusted and validated before it can affect application state.

Input	Validation applied
Question pack	Envelope checked (app, type, formatVersion, id, name); each question validated individually; malformed questions dropped; a pack with none left is rejected
Pack id or text clash	Rejected rather than overwritten - a pack can only add, never redefine an existing question
Pack size	Capped at 5,000 questions and 5 MB to avoid locking up a phone browser
Imported PDF	Size-capped; page count capped; parsed result presented for review before anything is stored
Restored backup	Validated in full before existing data is touched, so a bad import cannot destroy progress
Stored records	Malformed rows dropped on load; settings merged over defaults and range-checked

15.2 Failure handling

- If IndexedDB is unavailable - private browsing being the usual cause - the app falls back to an in-memory store and tells the user on the Home screen that progress will not persist. It does not silently appear to save.
- Opening the database has a three-second timeout, because Safari has been observed to leave the request pending in private mode.
- Service-worker registration failure is caught and logged; it costs offline support for that visit but does not break the application.
- A render error anywhere in the tree is caught by an error boundary that offers a reload, rather than showing a blank page.
- One malformed question cannot break the bank: runtime validation filters it out and the rest of the application continues.

15.3 Privacy position

- No account, email, phone number or profile. No user identity exists in the system.
- No analytics, telemetry, advertising or tracking of any kind.
- No third-party requests: system fonts, inline SVG icons, nothing loaded from a CDN.
- No secrets exist in the repository, because there is no service to authenticate against.
- All user data remains on the device; export and import are the only ways it moves, and the user initiates both.

Deployment & Usage

16.1 Build output

The production build is 19 static files. There is no server-side code, no environment variable and no secret. The PDF reader is listed separately because it is fetched only if a user imports a PDF.

File	Raw	Gzip
assets/pdf.worker.min-CeO1tn2L.js	1186.4 KB	372.5 KB
assets/pdf-DRTcSZ5S.js	463.0 KB	138.3 KB
assets/questions-C7duxXLC.js	262.5 KB	66.2 KB
assets/vendor-Dbu2L4y1.js	160.7 KB	52.5 KB
assets/index-DkeV_NSA.js	82.2 KB	26.0 KB
assets/style-C0Oy1Pqb.css	22.7 KB	4.8 KB
icons/icon-512.png	22.0 KB	18.9 KB
icons/maskable-512.png	16.1 KB	11.5 KB
icons/icon-192.png	7.5 KB	7.4 KB
packs/current-affairs-2026-08.json	7.2 KB	1.9 KB
icons/maskable-192.png	5.3 KB	5.0 KB
icons/apple-touch-icon.png	4.9 KB	4.5 KB
favicon.ico	4.0 KB	4.0 KB
sw.js	3.6 KB	1.5 KB
sample-questions.pdf	2.3 KB	0.9 KB
index.html	1.7 KB	0.7 KB
manifest.webmanifest	0.9 KB	0.3 KB
packs/index.json	0.4 KB	0.2 KB
assets/pdf.worker.min-q3wGjwao.js	0.1 KB	0.1 KB

Measure	Raw	Gzip
Total build output	2253.5 KB	717.3 KB
PDF reader (on demand only)	1649.5 KB	510.9 KB
Initial download	604.0 KB	206.4 KB

16.2 Hosting configuration

Setting	Value
---------	-------

Framework preset	Vite
Build command	npm run build
Output directory	dist
sw.js cache header	public, max-age=0, must-revalidate
index.html cache header	public, max-age=0, must-revalidate
/assets and /icons	public, max-age=31536000, immutable (filenames are content-hashed)
/packs	public, max-age=300, must-revalidate

Two configuration details that determine whether updates reach users

sw.js must be served with must-revalidate. Without it the host caches the service worker aggressively and application updates never reach a phone that has already installed it. Separately, cleanUrls is deliberately disabled: it makes /index.html issue a redirect, and a redirect during service-worker precaching silently breaks offline support.

16.3 Installation

The application is added to the Home Screen from the browser. It is not an App Store application and does not present itself as one.

Platform	Steps
iPhone	Open the URL in Safari (other iOS browsers cannot install) - Share - Add to Home Screen - Add
Android	Open in Chrome - menu - Install app

16.4 Commands

Command	Purpose
npm run dev	Development server with hot reload; service worker disabled
npm run build	Type-check and produce the production build
npm run serve	Serve the production build as a static host would
npm test	Run all 166 tests
npm run validate:questions	Validate the question bank against every quality gate
npm run verify	Validation, tests and build in sequence
npm run rebalance:answers	Even out which option position holds the correct answer
npm run pack:build	Build and index a downloadable question pack
npm run icons	Regenerate every application icon from code
npm run report	Regenerate and check this report

SECTION 17

Role & Contribution

My role on this project was owner and director: I defined what was to be built, set the constraints that shaped every technical decision, judged the output against those constraints, and redirected the work when it did not meet them. Implementation was carried out by an AI coding agent under that direction, as described in Section 11.

17.1 Contributions

Area	Contribution
Product definition	Defined the product, its user, and the daily-practice model it had to support
Constraints	Set the non-negotiables - no native app, no backend, no paid service, no account, offline-capable - which determined the architecture
Scope management	Decided what was built at each stage and what was deferred; added PDF import once the pack system proved the extension model
Technical direction	Rejected native packaging after reviewing its real cost; required the project stay a plain static site when the hosting target changed
Quality bar	Required that statistics never mislead, that content be verified rather than recalled, and that no capability be claimed without being demonstrated
Validation	Reviewed delivered output against the requirements and rejected work that did not meet them
Documentation	Directed the scope, structure and honesty standard of this report

17.2 Evidence of direction shaping the result

The clearest evidence is in the defects in Section 13. The answer-position bias, the contrast failures and the silently dropped PDF questions were all found because the work was held to a standard of demonstrated correctness rather than accepted output. Each produced a permanent control - a validator gate, a measurement step, a regression test - rather than a one-off fix.

Results & Verified Metrics

18.1 Objectives against outcome

#	Objective	Outcome
O1	Installable on iPhone without native development	Met. Manifest valid, standalone display, apple-touch-icon present. Not verified on physical hardware (Section 19).
O2	Fully functional offline	Met and verified: app loaded and ran quizzes with the server stopped
O3	At least 500 questions	Met: 602 built in, +10 in a pack
O4	Adaptive practice	Met and tested: weak categories take a larger share while all categories remain present
O5	Stable daily challenge	Met and tested: identical within a day, different across days
O6	Statistics never mislead	Met and tested: no percentage below 5 attempts; weak area requires 10
O7	Bank extensible after release	Met and verified: pack install and PDF import both exercised end to end
O8	Zero cost, no account	Met: no backend, no paid service, no credentials in the repository
O9	Correctness demonstrated	Met: 166 automated tests plus browser verification of each claim

18.2 Measured figures

Metric	Value
Questions (bundled / total available)	602 / 612
Categories	14, all populated
Difficulty split (Easy / Medium / Hard)	39.0% / 46.8% / 14.1%
Answer-position spread (A / B / C / D)	25.1% / 25.1% / 24.9% / 24.9%
Automated tests	166 across 8 suites, all passing
Source files / lines	47 / 7,559
Test code lines	2,028
Runtime dependencies	4
Build output	19 files, 717.3 KB gzip total
Initial download (excluding on-demand PDF reader)	206.4 KB
Contrast pairs below WCAG AA	0 of 16 measured, in both themes

Limitations

Stated directly, because a report that lists only successes is not useful for judging the work.

19.1 Not implemented

- No optical character recognition. A scanned or photographed PDF has no text layer and cannot be imported. The application detects this and says so rather than failing silently.
- No live current-affairs feed. Current-affairs questions are static and date-stamped; new ones arrive as packs, which keeps a person in the loop on factual accuracy.
- No cloud synchronisation. Progress is per-device; export and import move it between devices manually.
- Imported questions must have exactly four options, because the interface renders four choices throughout. Five-option questions are reported and skipped rather than truncated.
- Haptics have no effect on iPhone, as Safari does not implement the Vibration API. The setting exists for Android and desktop and the interface says so.
- The application is not distributed through the App Store and does not claim to be.

19.2 Not tested

- The application has not been run on physical iPhone hardware. Safe-area CSS, viewport-fit and the iOS meta tags are all present and the layout was verified at 390 x 844, but real Dynamic Island and home-indicator rendering has not been observed.
- The MCQ parser was tested against a purpose-built PDF and a broad set of text fixtures, not against a large corpus of real question papers. An unanticipated layout may parse poorly, which is why the review screen exists.
- Long-run spaced-repetition behaviour over weeks has been verified through unit tests on the interval logic, not by elapsed-time observation.
- No load or performance testing beyond build-size measurement; it is a single-user client-side application, so there is no concurrency to test.

Future Improvements

Practical next steps from the current state, in rough order of value relative to effort.

Improvement	Rationale	Effort
Verify on physical iPhone hardware	Closes the largest outstanding verification gap; safe-area and standalone rendering can only be confirmed on a device	Low
Expand the question bank	The loader picks up new category files automatically, so growth needs no code change	Ongoing
Support 2-6 option questions	Would let the importer accept five-option papers currently skipped; requires touching the quiz UI and validator together	Medium
Regular current-affairs packs	The publishing pipeline exists; this is a content cadence rather than an engineering task	Low per pack
Exam-specific modes	Category weighting per examination syllabus, reusing the existing selection engine	Medium
Optional OCR for scanned PDFs	Would remove the main import limitation, but adds a large dependency; best loaded on demand like pdf.js	High
Encrypted backup file	Export is plain JSON today; useful if backups are shared across devices via cloud storage	Low

If a server component is ever added, the architecture keeps that change contained: services are already isolated behind their own modules. Any such addition must keep secrets server-side, as the current build holds none.

Conclusion

GK Master delivers a daily general-knowledge practice tool that meets its constraints: it installs to a phone Home Screen without native development, works with no network connection, costs nothing to run, and holds no user account or personal data. The question bank is large enough for sustained use and can grow after release through two independent routes, neither of which requires rebuilding the application.

The parts of the work that proved most demanding were not the interface but the correctness properties behind it: making the daily challenge reproducible, making practice adapt without collapsing onto one topic, making statistics refuse to mislead, and parsing question papers without inventing answers. Several defects in these areas were found only because each claim was checked rather than assumed, and each fix left behind a permanent control rather than a patch.

The remaining gap is verification on physical iPhone hardware, and the main functional limitation is that scanned PDFs cannot be imported. Both are recorded in Section 19 rather than omitted.

Master Prompt

A single prompt that rebuilds this entire project from an empty directory. It carries the constraints, the architecture, the algorithms, the quality gates and the verification standard that produced what this report describes, including the defects in Section 13 stated as requirements so they are not repeated.

The same specification is given below in six prompt structures. The substance is identical in each; only the scaffolding differs, because different models follow different structures more reliably. Use whichever matches the model you intend to run, and paste the complete block as a single prompt.

#	Structure	Shape
22.1	Claude	XML-tagged sections. Claude follows explicit tag boundaries well, and the final_check block gives it a self-review step.
22.2	OpenAI (general)	Role, personality, goal, success criteria and explicit stop rules — suited to agentic runs where the model decides when it is finished.
22.3	GPT-4.1	Headed sections with sub-categories, explicit reasoning steps and worked examples, which this family follows more literally.
22.4	Codex	Objective, current state, requirements, acceptance criteria and verification — the shape a coding agent can check itself against.
22.5	Gemini	Instructions and constraints stated before the long context block, with a single closing instruction carrying the priority.
22.6	Universal	Model-agnostic ordering: role, objective, context, requirements, constraints, success criteria, examples, output, verification.

What the prompt deliberately constrains

The prompt states not only what to build but what to refuse: no invented answers, no fabricated statistics, no claimed deployment without a verified URL, and no statistic shown without enough data behind it. Those constraints are what kept this build honest, so they belong in the prompt as much as the feature list does.

22.1 Claude

XML-tagged sections. Claude follows explicit tag boundaries well, and the final_check block gives it a self-review step.

```
<role>
You are a senior full-stack engineer, PWA specialist, educational-product
designer, and QA engineer. You own this build end to end.
</role>

<context>
You are building a complete, production-quality application from an empty
directory. Node 20 or newer is available. There is no existing code.
The user wants a daily general-knowledge trainer they can install on an iPhone
without native development, an App Store account, or any running cost.
</context>

<instructions>
1. Inspect the working directory and toolchain before creating anything.
2. Build the product exactly as specified in <input>.
3. Make product and technical decisions yourself; do not ask which colour,
   framework or layout to use.
4. Write the question bank yourself, verifying facts rather than recalling them.
5. Add automated tests alongside the implementation, not afterwards.
6. Verify the finished build in a real browser before reporting completion.
```

7. Report honestly what works, what does not, and what you did not test.
</instructions>

<constraints>

- No backend, no database server, no paid service, no API key, no login.
- No analytics, advertising, or third-party network requests.
- Output must be static files that work offline and from any sub-path.
- Never invent an answer, a statistic, or a deployment URL.
- Never present a number the underlying data does not support.

</constraints>

<input>

PRODUCT

Name: GK Master. Tagline: "Train your knowledge. Every single day."

A personal General Knowledge trainer for daily practice, used mainly on a phone.

Primary user: one individual studying general knowledge, often on an iPhone, frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS

- Progressive Web App only. No native app, no Xcode, no App Store, no paid developer account.
- No backend, no database server, no serverless functions. The build output must be static files servable from any host.
- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server rewrite rules.

TECHNOLOGY

TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS driven by design tokens: no CSS framework, no component library, no icon package (draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb. Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE

Keep every algorithm in React-free service modules so they are unit-testable in isolation:

src/services/quiz/selection.ts	adaptive and deterministic question selection
src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.
- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy})/70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.
- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes.

Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support.

iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target.

Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options,

one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals, streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance.

Write a README covering setup, development, deployment, question format, backup and limitations.

Do not claim anything is deployed unless the URL has been opened and checked.

State plainly what was not implemented and what was not tested.

</input>

<output_format>

A complete repository containing source, question data, tests, build scripts and a README. Finish with a short report stating: what was built, test results, build size, what was verified in a browser, and known limitations.

</output_format>

<final_check>

Before finishing, confirm: validation passes; all tests pass; the production build succeeds; the service worker registers and the app works offline; no single answer position exceeds 35%; no statistic is shown without enough data; and no claim in your report is unverified.

</final_check>

22.2 OpenAI (general)

Role, personality, goal, success criteria and explicit stop rules — suited to agentic runs where the model decides when it is finished.

Role: Senior full-stack engineer delivering a complete offline-first Progressive Web App from an empty directory, with no backend and no running cost.

Personality

Decisive and self-directed. Makes product and design calls without asking. Reports progress plainly, including failures. Never overstates what is finished.

Goal

A daily general-knowledge trainer the user can open in Safari, add to the iPhone Home Screen, and use offline, with no account and no subscription.

Success criteria

- Production build is static files only and runs offline after first load.
- 500+ verified questions across 14 categories, each with four options and an explanation, with correct answers spread evenly across A/B/C/D.
- Daily challenge is identical on repeat visits the same day, new the next day.
- Spaced repetition returns missed questions and retires mastered ones.
- Statistics are withheld until the data supports them.
- 150+ automated tests pass; the build is verified in a real browser.

Constraints

- No backend, database server, paid API, API key, login, analytics or ads.
- No third-party network requests; system fonts and inline SVG icons only.
- Must work from a domain root or a sub-path without server rewrite rules.
- Never fabricate an answer, a metric, or a deployment URL.

Specification

PRODUCT

Name: GK Master. Tagline: "Train your knowledge. Every single day."

A personal General Knowledge trainer for daily practice, used mainly on a phone.

Primary user: one individual studying general knowledge, often on an iPhone, frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS

- Progressive Web App only. No native app, no Xcode, no App Store, no paid developer account.
- No backend, no database server, no serverless functions. The build output must be static files servable from any host.
- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server rewrite rules.

TECHNOLOGY

TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS driven by design tokens: no CSS framework, no component library, no icon package (draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb. Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE

Keep every algorithm in React-free service modules so they are unit-testable in isolation:

src/services/quiz/selection.ts	adaptive and deterministic question selection
src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.
- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy})/70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.
- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes.

Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support. iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target. Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options, one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals, streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server

stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance.
Write a README covering setup, development, deployment, question format, backup and limitations.
Do not claim anything is deployed unless the URL has been opened and checked.
State plainly what was not implemented and what was not tested.

Output

A complete repository plus a concise completion report: what was built, test results, bundle size, browser verification performed, and known limitations.

Stop rules

- If a required fact cannot be verified, drop it rather than guessing.
- If deployment requires credentials you do not have, stop at that exact point, say so, and give the single command the user must run.
- Do not report completion while any test fails or the build is broken.

22.3 GPT-4.1

Headed sections with sub-categories, explicit reasoning steps and worked examples, which this family follows more literally.

```
# Role and Objective
You are a senior full-stack engineer. Build a complete, offline-first
Progressive Web App called GK Master: a personal daily general-knowledge trainer
that installs to an iPhone Home Screen with no native code and no running cost.

# Instructions
- Build the entire product from an empty directory, including the question bank.
- Decide framework details, visual design and file layout yourself.
- Write tests alongside the implementation.
- Verify the result in a real browser before declaring completion.

## Sub-categories
### Product and constraints
- Static output only: no backend, no database server, no paid service, no login.
- Offline-capable, installable, portable across hosts and sub-paths.
### Content
- 500+ verified questions across 14 fixed categories, four options each, with an
  explanation, difficulty, and a distinct id range per category.
- Correct answers spread evenly across option positions and enforced by a test.
### Behaviour
- Deterministic daily challenge, adaptive selection, spaced repetition, streaks.
- Statistics withheld until there is enough data to justify them.
### Extensibility
- Downloadable question packs and in-browser PDF import, both validated and
  reviewable before anything enters the bank.

# Reasoning Steps
1. Inspect the environment and available toolchain.
2. Scaffold the project and the type contracts first.
3. Implement the React-free service layer (selection, revision, statistics,
  streaks, storage) and test it in isolation.
4. Build the UI on top of that layer.
5. Author and validate the question bank; fix any answer-position skew.
6. Add the PWA layer, then the pack and PDF import systems.
7. Run validation, tests and build; then verify behaviour in a browser.

# Specification
PRODUCT
Name: GK Master. Tagline: "Train your knowledge. Every single day."
A personal General Knowledge trainer for daily practice, used mainly on a phone.
Primary user: one individual studying general knowledge, often on an iPhone,
frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS
- Progressive Web App only. No native app, no Xcode, no App Store, no paid
  developer account.
- No backend, no database server, no serverless functions. The build output must
  be static files servable from any host.
```

- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server rewrite rules.

TECHNOLOGY

TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS driven by design tokens: no CSS framework, no component library, no icon package (draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb. Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE

Keep every algorithm in React-free service modules so they are unit-testable in isolation:

src/services/quiz/selection.ts	adaptive and deterministic question selection
src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.

- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy})/70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.
- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes. Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support. iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target. Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options, one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question

so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals, streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance. Write a README covering setup, development, deployment, question format, backup and limitations. Do not claim anything is deployed unless the URL has been opened and checked. State plainly what was not implemented and what was not tested.

Output Format

A complete repository with source, data, tests, scripts and README, followed by a completion report: build result, test count, bundle size, browser verification, and limitations.

Examples

Example 1

Context: a category has 3 recorded attempts.

Correct output: the UI shows "Building data", not a percentage.

Example 2

Context: a PDF question has options but no marked answer.

Correct output: the question is listed, flagged, and skipped. Never guessed.

Final instructions

Do not claim completion while a test fails. Do not claim a deployment without opening the URL. State plainly what was not implemented and what was not tested.

22.4 Codex

Objective, current state, requirements, acceptance criteria and verification — the shape a coding agent can check itself against.

Objective

Build GK Master: a complete, offline-first Progressive Web App for daily general-knowledge practice, installable to an iPhone Home Screen, with no backend and no running cost.

Context

Personal learning tool for a single user who studies general knowledge daily, most often on a phone and often without a reliable connection. It must be installable without native development, an App Store account, or any paid service.

Current State

Empty directory. Node 20 or newer available. No existing code, data or assets.

Requirements

PRODUCT

Name: GK Master. Tagline: "Train your knowledge. Every single day."

A personal General Knowledge trainer for daily practice, used mainly on a phone.

Primary user: one individual studying general knowledge, often on an iPhone, frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS

- Progressive Web App only. No native app, no Xcode, no App Store, no paid developer account.
- No backend, no database server, no serverless functions. The build output must be static files servable from any host.
- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server rewrite rules.

TECHNOLOGY

TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS driven by design tokens: no CSS framework, no component library, no icon package (draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb. Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE

Keep every algorithm in React-free service modules so they are unit-testable in isolation:

src/services/quiz/selection.ts	adaptive and deterministic question selection
src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app

can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.
- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy})/70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.
- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes. Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support. iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target. Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options, one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals, streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance.

Write a README covering setup, development, deployment, question format, backup and limitations.

Do not claim anything is deployed unless the URL has been opened and checked.

State plainly what was not implemented and what was not tested.

Constraints

- * TypeScript, React 18 and Vite 5; no CSS framework, component library or icon package.
- * No backend, database server, paid API, API key, login, analytics or advertising.
- * No third-party network requests at runtime.
- * Build output must be static and work from any sub-path without rewrite rules.
- * No secrets anywhere in the repository.

Acceptance Criteria

- * npm run validate:questions passes, including the answer-position skew limit.
- * 150+ tests pass across unit and end-to-end UI suites.
- * npm run build produces static files only.
- * The service worker registers, activates and serves the app with the server stopped.
- * Reopening the app on the same day returns the identical daily challenge; the next day returns a different one.
- * Always selecting a single option letter scores near 25%.
- * Categories with fewer than 5 attempts display a label, not a percentage.
- * A PDF question with no marked answer is reported and skipped, never guessed.

Verification

- * Run npm run verify (validation, tests, build).
- * Serve the production build and confirm in a real browser: service worker active, offline operation with the server stopped, manifest and all icons loading, progress surviving a reload, and both themes applying.

- * Serve the same build from a sub-path and confirm assets and worker scope resolve.
- * Measure contrast for every colour pair in both themes; correct anything below 4.5:1.

Deliverable

A complete repository with source, question data, tests, tooling scripts and a README, plus a completion report stating test results, bundle size, what was verified in a browser, and known limitations.

22.5 Gemini

Instructions and constraints stated before the long context block, with a single closing instruction carrying the priority.

```
<role>
You are a senior full-stack engineer and PWA specialist building a complete
application from an empty directory.
</role>

<instructions>
1. Build the product described in <context> in full, including its question bank.
2. Keep all algorithms in React-free modules so they can be unit-tested alone.
3. Write automated tests alongside the implementation.
4. Validate the question data with a script that fails the build on bad data.
5. Verify the finished build in a real browser before reporting completion.
6. Report honestly, including what you did not implement or test.
</instructions>

<constraints>
- No backend, database server, paid API, API key, login, analytics or ads.
- Static output only; must work offline and from any sub-path.
- Never invent an answer, a statistic, or a deployment URL.
- Never show a number the underlying data does not support.
</constraints>

<output_format>
A complete repository (source, data, tests, scripts, README) plus a short
completion report covering test results, bundle size, browser verification and
limitations.
</output_format>

<context>
PRODUCT
Name: GK Master. Tagline: "Train your knowledge. Every single day."
A personal General Knowledge trainer for daily practice, used mainly on a phone.
Primary user: one individual studying general knowledge, often on an iPhone,
frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS
- Progressive Web App only. No native app, no Xcode, no App Store, no paid
  developer account.
- No backend, no database server, no serverless functions. The build output must
  be static files servable from any host.
- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no
  third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server
  rewrite rules.

TECHNOLOGY
TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS
driven by design tokens: no CSS framework, no component library, no icon package
(draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb.
Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE
Keep every algorithm in React-free service modules so they are unit-testable in
isolation:
  src/services/quiz/selection.ts    adaptive and deterministic question selection
```

src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.
- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy}) / 70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a

category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.

- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes. Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support. iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target. Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options, one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals,

streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance.

Write a README covering setup, development, deployment, question format, backup and limitations.

Do not claim anything is deployed unless the URL has been opened and checked.

State plainly what was not implemented and what was not tested.

</context>

<task>

Implement the entire specification above, then run validation, tests and the production build, and verify the result in a browser.

</task>

<final_instruction>

Correctness of the question data and honesty of the statistics matter more than feature count. If a fact cannot be verified, drop it.

</final_instruction>

22.6 Universal

Model-agnostic ordering: role, objective, context, requirements, constraints, success criteria, examples, output, verification.

ROLE

Senior full-stack engineer, PWA specialist and QA engineer, owning the build end to end.

OBJECTIVE

Deliver GK Master, an offline-first Progressive Web App for daily general-knowledge practice, installable to a phone Home Screen, with no backend and no running cost.

CONTEXT

Empty directory, Node 20+. Single user, phone-first, frequently offline. No native development, no App Store account, no paid services.

REQUIREMENTS

PRODUCT

Name: GK Master. Tagline: "Train your knowledge. Every single day."

A personal General Knowledge trainer for daily practice, used mainly on a phone.

Primary user: one individual studying general knowledge, often on an iPhone, frequently without a reliable connection.

NON-NEGOTIABLE CONSTRAINTS

- Progressive Web App only. No native app, no Xcode, no App Store, no paid developer account.
- No backend, no database server, no serverless functions. The build output must be static files servable from any host.
- No paid APIs, no API keys, no secrets of any kind in the repository.
- No account, no login, no user identity, no analytics, no advertising, no third-party network requests (system fonts only, no CDN).
- Must work fully offline after first load and be installable to the Home Screen.
- Must run correctly from a domain root AND from a sub-path with no server rewrite rules.

TECHNOLOGY

TypeScript, React 18, Vite 5. Hash-based routing for host portability. Plain CSS driven by design tokens: no CSS framework, no component library, no icon package (draw icons as inline SVG paths). Vitest + Testing Library + fake-indexeddb. Keep runtime dependencies minimal and be able to justify each one.

ARCHITECTURE

Keep every algorithm in React-free service modules so they are unit-testable in isolation:

src/services/quiz/selection.ts	adaptive and deterministic question selection
src/services/quiz/session.ts	session lifecycle and scoring
src/services/revision/	spaced repetition
src/services/statistics/	accuracy, category and difficulty statistics
src/services/streak/	daily streak rules
src/services/storage/	IndexedDB wrapper, backup and restore
src/services/packs/	downloadable question packs
src/services/import/	PDF text extraction and MCQ parsing
src/data/questions/*.json	question bank, one file per category
src/data/questionBank.ts	loads, validates and indexes the bank

React layer: components/, features/, pages/, and one context store in hooks/.

SCREENS

Bottom navigation: Home, Practice, Progress, Settings. The quiz is a separate full-screen view with no navigation bar.

- Welcome: shown once on first launch. No registration.
- Home: brand header, streak badge, hero card for the daily challenge with progress, quick stats, focus area, revision prompt.
- Quiz: "Question N of M" counter with progress bar, category and difficulty badges, four large option cards, immediate lock-in on selection, explanation, Next button disabled until answered.
- Results: score ring, encouraging message banded by score, correct/incorrect counts, review of mistakes.
- Practice: session modes (random, weak areas, revision, quick 5, deep 50) and a category grid showing per-category accuracy.
- Progress: overall and recent accuracy, per-category mastery, per-difficulty breakdown, current and best streak, mastered count.
- Settings: questions per day (5/10/20/30, default 20), theme (system/light/dark, default dark), haptics, sound, question updates, export/import/reset, about, and an in-app install guide.

QUESTION BANK

Author at least 500 factually accurate, unambiguous questions across exactly these 14 categories: Indian History, World History, Geography, Indian Polity, Economy, Science, Technology, Sports, Art & Culture, Books & Authors, India GK, World GK, Current Affairs, Miscellaneous GK.

Schema per question: { id, question, options[4], correctAnswer (0-based index), explanation, category, difficulty (Easy|Medium|Hard), tags[], source, sourceUrl, publishedDate }. Reserve a distinct id range per category. Target roughly 40% Easy / 40% Medium / 20% Hard.

Spread the correct answer evenly across positions A/B/C/D. Hand-authored banks drift heavily toward one letter; if any single position exceeds about 30% the app can be beaten by always choosing it. Provide a rebalancing tool, enforce the limit in the validator, and assert it in a test.

Current Affairs questions must carry a real publishedDate that the UI displays, so dated facts are never presented as live news. Verify current-affairs facts against primary sources and drop anything that cannot be confirmed.

LEARNING ENGINE

- Daily challenge must be deterministic: seed a PRNG (FNV-1a hash feeding Mulberry32) from the LOCAL calendar date so reopening the app on the same day returns the identical set, and a new day returns a new one. Allocate category quotas with the largest-remainder method so allocation is reproducible. Resume an in-progress challenge rather than regenerating it.
- Adaptive selection: category weight = $1 + \max(0, 70 - \text{accuracy})/70 * 2$, never zero, so weak categories are drilled harder while strong ones keep appearing. Give untouched categories a small boost. Avoid the ~120 most recently served questions and hold back mastered ones, using tiers that relax only when a tier runs dry so a session is always full length.
- Difficulty mix follows recent accuracy: 50/35/15 below 60%, 40/40/20 as the baseline, 30/45/25 above 75%, 20/45/35 above 85%.
- Spaced repetition: mastery level 0-5 with review intervals [0,1,3,7,16,35] days. A correct answer advances one level; an incorrect answer drops two. A missed question returns the next day, never the same day. Reaching the top level retires a question from the mistake queue permanently. Order the revision queue by overdue first, then repeat offenders, then most recently missed.
- Streaks: increment once per calendar day, reset after a missed day, and compute

the displayed streak separately from the stored one so a stale streak shows zero.

HONESTY RULES (treat as functional requirements)

- Never display a statistic that the data does not support. Under 5 attempts in a category show "Building data"; zero attempts show "Not started"; only beyond that show a percentage.
- Require at least 10 attempts before naming a category as the weak area, so a single wrong answer cannot brand a topic weak. If nothing qualifies, say so.
- Never invent an answer, a statistic, or a deployment URL.

PROGRESSIVE WEB APP

manifest.webmanifest with standalone display, portrait orientation, relative scope and start_url, and 192/512 icons in both "any" and "maskable" purposes. Hand-write the service worker and template it at build time with a small Vite plugin that injects the precache manifest and derives the cache version from the content hashes of the shipped files, so any change retires the previous cache. Network-first for navigations, cache-first for content-hashed assets, and an honest error for an uncached asset that cannot be fetched. Add precache entries individually so one failed asset cannot break offline support. iPhone specifics: viewport-fit=cover with env(safe-area-inset-*), apple-touch-icon, black-translucent status bar, and a 48px minimum touch target. Generate every icon from code (signed-distance-field monogram plus a hand-written PNG encoder using only Zlib) rather than committing opaque binaries.

GROWING THE BANK AFTER RELEASE

- 1) Question packs. JSON envelope { app, type, formatVersion, id, name, createdAt, questions[] } plus a pack index document. Settings can check a configurable source, list packs not yet installed, install them, and remove them again. Validate every pack; reject any question colliding with an existing one by id OR by normalised question text, so a pack can only ever add.
- 2) PDF import. Read multiple-choice questions out of a PDF entirely in the browser using pdf.js, loaded lazily and excluded from the precache because it is large. Rebuild lines from positioned text fragments by grouping on baseline. Recognise: "1." "1)" "Q1." "Q.1" numbering; "(a)" "a)" "A." "[A]" "1)" options, one per line or several on one line; "Answer: B" / "Ans - (b)" inline or a trailing ANSWER KEY; "Explanation:" blocks. Rejoin wrapped stems and options, strip page numbers, and remove running headers using PAGE BOUNDARIES rather than repetition counts. Track whether an option run uses letters or digits, or question 5 following options A-D will be absorbed as a fifth option. Show a review screen before importing: list everything found, flag questions with no answer and skip them rather than guessing, let the user choose category and difficulty and exclude individual items. Detect PDFs with no text layer and explain that clearly instead of failing silently. Do not add OCR.

DATA AND PRIVACY

IndexedDB with object stores for settings/streak, per-question progress, sessions, and installed packs. Fall back to an in-memory store if IndexedDB is unavailable and tell the user progress will not persist rather than pretending to save. Time out the database open so a hung request cannot block startup. Drop malformed rows on load. Provide JSON export and import, validating a backup before touching any existing data. Reset requires explicit confirmation naming what will be deleted, and preserves the question bank.

ACCESSIBILITY

Convey answer correctness through colour AND icon AND text, never colour alone. Use semantic HTML with labelled controls, radiogroups for segmented choices, and role="switch" for toggles. Move focus to the question heading on each new question so screen readers announce it, but suppress the focus ring on non-interactive elements. Announce explanations via an aria-live region. Respect prefers-reduced-motion. Measure the contrast ratio of every colour pair in both themes and correct anything below WCAG AA 4.5:1 rather than assuming it passes.

TOOLING

Provide scripts for: question validation (unique ids, unique text, exactly four non-empty distinct options, answer index in range, explanation present, known category and difficulty, dated current affairs, answer-position skew limit, minimum total), answer-position rebalancing, icon generation, pack building, a static preview server that can also serve from a sub-path, and screenshot capture. Provide one "verify" script chaining validation, tests and build.

TESTING

Cover at minimum: daily-challenge determinism (same day identical, next day different), category weighting, difficulty mix, quota allocation, recency avoidance, short-pool behaviour, spaced-repetition transitions and intervals, streak rules including staleness, statistics minimum-data guards, session scoring, storage round-trips against a real IndexedDB implementation, backup validation and rejection of invalid files, pack install and duplicate rejection, MCQ parsing across every supported layout, and a full end-to-end UI flow from onboarding to results and persistence. Aim for 150 or more tests.

DELIVERY AND VERIFICATION

Build, run the tests, then verify in a real browser rather than by assertion: the service worker registers and activates; the app still works with the server stopped; the manifest and every icon load; progress survives a reload; both themes apply; and always choosing a single option letter scores near chance.

Write a README covering setup, development, deployment, question format, backup and limitations.

Do not claim anything is deployed unless the URL has been opened and checked.

State plainly what was not implemented and what was not tested.

CONSTRAINTS

- Static build output only; no backend, database server or serverless function.
- No paid API, API key, login, analytics, advertising or third-party request.
- Must run from a domain root or a sub-path with no server rewrite rules.
- No secrets in the repository.

SUCCESS CRITERIA

- Validation, 150+ tests and the production build all pass.
- App works offline with the server stopped.
- Daily challenge is stable within a day and changes the next day.
- No answer position exceeds 35% of correct answers.
- Statistics are withheld until the data supports them.
- Every colour pair meets WCAG AA 4.5:1 in both themes.

EXAMPLES

- A category with 3 attempts shows "Building data", not 33%.
- A PDF question with no marked answer is flagged and skipped, never guessed.
- A pack question whose text already exists is rejected, not duplicated.

OUTPUT FORMAT

Complete repository plus a concise completion report: build result, test count, bundle size, browser verification performed, and known limitations.

VERIFICATION / STOP CONDITIONS

- Run validation, tests and build; do not report completion if any fails.
- Verify offline operation, manifest, icons, persistence and theming in a real browser.
- If deployment needs credentials you do not have, stop there and state the exact command the user must run. Never invent a URL.

22.7 Notes on use

- Paste one complete block as a single prompt. Each is self-contained; nothing else from this report is needed alongside it.
- Expect the agent to make product and visual decisions independently. The prompts fix the constraints, the algorithms and the verification standard, and leave presentation open.
- The four parts that matter most are the honesty rules, the answer-position requirement, the determinism requirement and the PDF parser pitfalls. Each encodes a defect that was actually encountered during this build.
- Deployment will still require a one-time login to the chosen host. No prompt removes that step, and the prompts instruct the agent to stop there rather than invent a URL.